

IA Aplicada com LLMs

Aula 05 — Padrões de arquitetura de agentes

Onde paramos

A Aula 04 apresentou uma dúzia de casos de indústria sob a pergunta "*que padrão é este?*". A resposta possível era um vocabulário **de nomes**.

E a Aula 03 encerrou com uma lista de pendências:

estado e memória · múltiplas ferramentas · outros padrões de arquitetura · MCP · confiabilidade · context engineering dinâmica

Hoje os nomes viram código, e quatro pendências são encerradas.

O laço entregue pela Aula 03

```
for passo in range(max_passos):  
    r = client.chat.completions.create(messages=mensagens, tools=DECLARACOES)  
    if not r.choices[0].message.tool_calls:  
        return r.choices[0].message.content  
    ...
```

Três limitações o impedem de operar sem supervisão:

1. **Não sabe parar** — `max_passos` é teto, não orçamento → nota 03
2. **Não sabe falhar** — uma exceção derruba o processo → nota 03
3. **Não sabe esquecer** — o histórico apenas cresce → nota 04

E uma questão anterior a todas

A tarefa exigia um laço?

Este deck trata dessa questão.

Com um case já escolhido, a pergunta pertinente em cada padrão não é de compreensão, e sim de aplicação: **é este o padrão que o case exige?**

Roteiro: a linha divisória · autópsia do exercício 03 ·
cinco padrões · a tabela de decisão · composição

Parte 1

A linha divisória

De definição a critério operacional

Workflow: o caminho está no código.

Agente: o caminho é decidido em execução pelo modelo.

Definição adequada para exposição, insuficiente para decisão de projeto.

É possível desenhar o fluxograma antes da execução?

Sim → escrever o fluxograma em Python.

Não → há motivo para autonomia.

Três perguntas de apoio

	Sim	Não
O nº de passos é conhecido?	workflow	agente
A ordem depende de dados de execução?	agente	workflow
O erro é caro ou irreversível?	reduzir a autonomia	tolerável

A terceira não trata da natureza da tarefa — trata da **consequência do erro**.

Tarefa que admitiria um agente, mas cujo erro emite dinheiro, é tarefa em que se aceita um sistema pior em troca de previsibilidade.

O preço de subir no espectro

Nível	Chamadas	Previsível	Depurável
Prompt único	1	sim	sim
Chain	N fixo	sim	sim
Roteador	1 + rota	sim	sim
Agente	?	não	apenas com trace
Multiagente	??	não	muito difícil

A última coluna é a mais onerosa: num agente a trajetória difere a cada execução e, sem log, resta uma saída incorreta sem indício de causa.

Parte 2

Autópsia do exercício 03

O workflow disfarçado de agente mais próximo

...foi produzido pela turma na aula anterior.

Etapa	Classificação
1. Coleta	workflow — a condição de parada é um <code>if</code>
2. Consulta	workflow — sempre, e nessa posição
3. Raciocínio	decisão — de uma, não de uma sequência
4. Ação	workflow condicional
5. Consulta final	workflow — sempre por último

Quatro das cinco etapas estavam no código

A sequência constava **numerada no enunciado**.

Isso não constitui erro: o objetivo daquele exercício era o **mecanismo**.

Em produção, porém, a versão em workflow seria superior em todos os eixos: menos chamadas, sem risco de omitir a consulta final ou de abrir dois chamados, testável etapa por etapa. Perderia apenas o atendimento fora do roteiro — e é nesse ponto que a decisão se coloca.

Quando o enunciado vem numerado, o código também deveria vir.

Parte 3

Os cinco padrões

Taxonomia de ANTHROPIC (2024)

1. Prompt chaining com portão

```
entrada —> [ LLM 1 ] —> portão —> [ LLM 2 ] —> saída
                        |
                        └─ falhou —> não segue adiante
```

```
bruto = chamar(PROMPT_EXTRACAO, texto, schema=SCHEMA_EXTRACAO)
if not bruto.get("valor") or bruto["valor"] <= 0:    # 0 PORTÃO:
    raise ValorInvalido(bruto)                    # código, não modelo
return chamar(PROMPT_PARECER, bruto, schema=SCHEMA_PARECER)
```

Regra expressável em Python é implementada em Python: mais barata, e não alucina.

Não usar: etapas interdependentes · nº de etapas dependente da entrada

2. Roteador — o modelo classifica, o código despacha

```
entrada —> [ classificador ] —> rota A: código puro (sem LLM)
                        |
                        |> rota B: outro prompt / outro modelo
                        |
                        |> rota C: humano
                        |
                        |> nenhuma —> fila de revisão
```

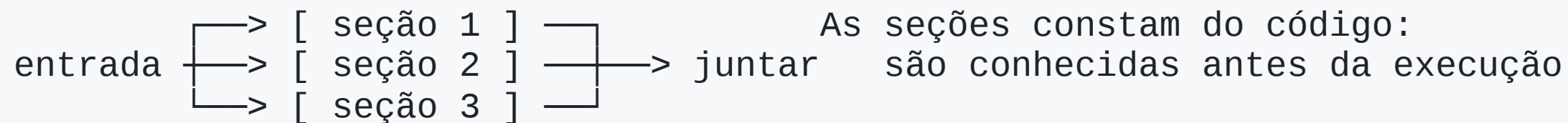
```
r = classificar(item, schema_com_enum(ROTAS))      # 1 chamada, barata
if r["rota"] == "dentro_da_politica":
    return aprovar_por_regra(item)                  # ZERO chamadas de LLM
if r["rota"] == "ambiguo": return agente_analisa(item)
if r["rota"] == "acima_da_alcada": return encaminhar_humano(item)
return fila_de_revisao(item)                        # "nenhuma": não inferir
```

A rota **nenhuma** é obrigatória — sem escape, o modelo elege opção inaplicável com alta confiança. **A de maior valor é a que não chama o modelo.**

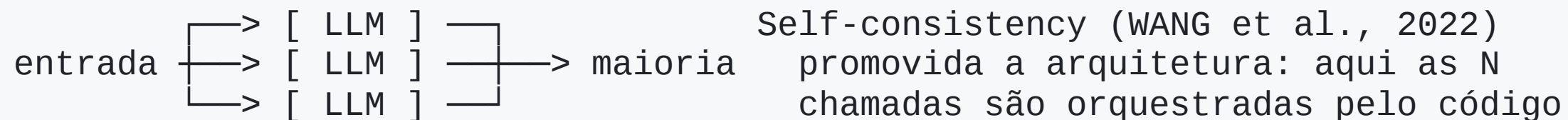
Não usar: rotas quase idênticas · classificação trivial em `if`

3. Paralelização — duas formas com o mesmo nome

Sectioning — subtarefas **independentes**, conhecidas de antemão



Voting — a *mesma* tarefa N vezes, decisão por maioria



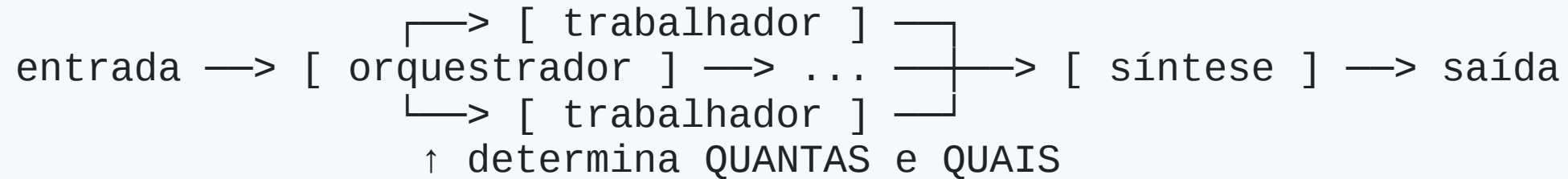
3. Voting: a divergência é informação

```
vereditos = [classificar(item) for _ in range(n)]
rota, votos = Counter(v["rota"] for v in vereditos).most_common(1)[0]
if votos < n // 2 + 1:
    return {"rota": "nenhuma", "motivo": "sem maioria"}
```

Três vereditos distintos **não** indicam que se deva eleger um deles.
Indicam que o caso é difícil — e, provavelmente, que cabe encaminhamento a humano. É o sinal de incerteza de menor custo disponível.

Não usar: *sectioning* com partes dependentes · *voting* em tarefa aberta ("redigir um resumo") — não há maioria entre três textos distintos

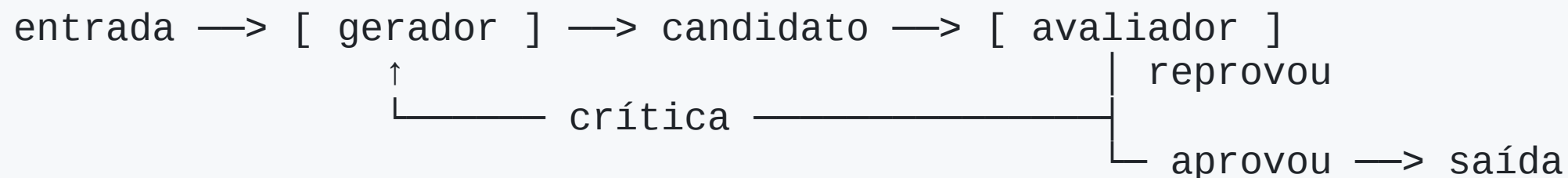
4. Orquestrador-trabalhador



```
plano = chamar(PROMPT_ORQUESTRADOR, lote, schema=SCHEMA_PLANO)
if len(plano["subtarefas"]) > MAX_SUBTAREFAS:      # orçamento, já aqui
    raise PlanoGrandeDemais(len(plano["subtarefas"]))
resultados = [executar_subtarefa(s) for s in plano["subtarefas"]]
```

Distinção única e decisiva em relação ao *sectioning*: **as subtarefas só existem em execução**. Sem teto, trata-se de conta aberta assinada por um modelo. **Em caso de dúvida entre os dois, prefere-se sectioning.**

5. Avaliador-otimizador (SHINN et al., 2023)



```
for rodada in range(max_rodadas):  
    candidato = chamar(PROMPT_GERADOR, entrada, critica=critica)  
    aval = chamar(PROMPT_AVALIADOR, candidato, schema=SCHEMA_AVALIACAO)  
    if aval["aprovado"]: return candidato, "aprovado"  
    critica = aval["o_que_corrigir"]  
return candidato, "teto_de_rodadas"      # melhor esforço, sinalizado
```

5. Duas armadilhas sistemáticas

Sem critério escrito, o avaliador produz elogio.

```
{"nota": 8}                                # não acionável  
{"cita_regra": false, "o_que_corrigir": "..."} # acionável
```

Sem teto de rodadas, o par oscila. O avaliador exige A, o gerador entrega A e perde B, o avaliador exige B. O retorno precisa **declarar** a saída por teto — melhor esforço apresentado como aprovação é pior que reprovação explícita.

Não usar: quando o critério não é escrevível.

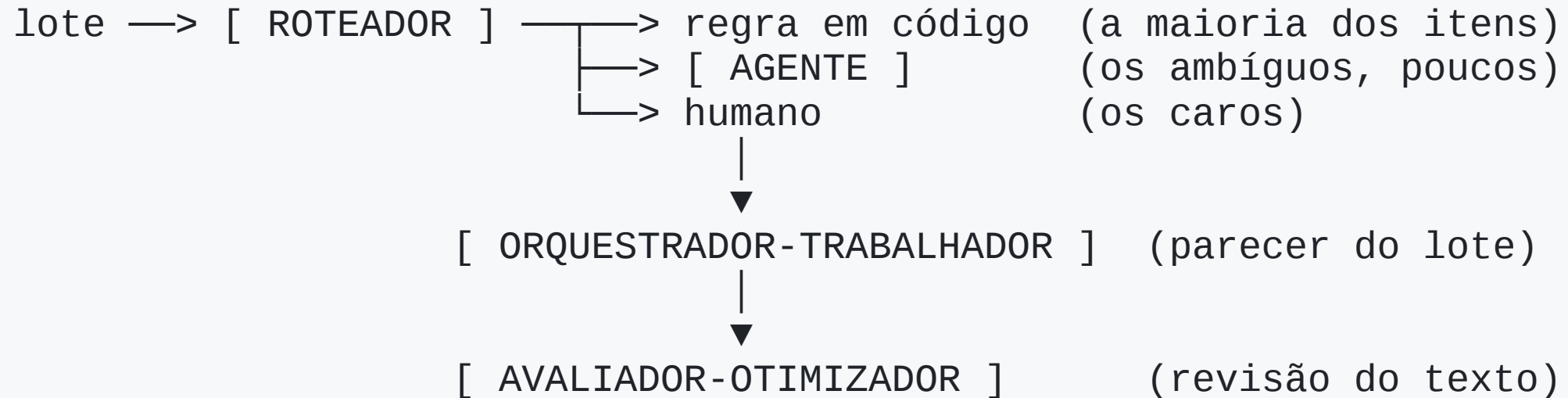
Nesse caso o avaliador adequado é uma pessoa.

A tabela de decisão

Padrão	Custo	Não usar quando
Prompt único	1	exige validação por outra chamada
Chaining + portão	N fixo	etapas interdependentes
Roteador	1 + rota	rotas quase idênticas
Sectioning	1 por seção	partes dependentes
Voting	3–5	tarefa aberta
Orquestrador	? — exige teto	as subtarefas são enumeráveis
Avaliador	2 por rodada	sem critério escrito
Agente	? — exige orçamento	o fluxograma existe

Leitura de cima para baixo, **interrompendo no primeiro que resolve.**

Composição de padrões



Este é o desenho do exercício desta aula: a autonomia cara fica confinada ao caminho estreito em que é necessária, e o volume trafega por código determinístico.

Autonomia é recurso escasso. Gasta-se onde produz retorno.

O exemplo que encerra o deck

1.000 despesas, 15% em violação da política:

Arquitetura	Conta	Chamadas
Agente por despesa	$1.000 \times \sim 4$ passos	~ 4.000
Chain fixa	$1.000 + 150$	1.150
Roteador + regra em código	$0 + 300 + 150$	450

Redução de quase nove vezes — e os 70% resolvidos por regra são **mais** confiáveis, uma vez que regra não alucina. O ganho não decorre de prompt melhor nem de modelo melhor, e sim da decisão sobre **o que não enviar ao modelo**.